

ASSIGNMENT 2 - Simple Unix Shell

Github Repository - https://github.com/dhruv230605/OS_Assignment_2

Implementation:

In this assignment we have created SimpleShell using the c language , which is an implementation of a simple Unix Shell. It provides the users with a command prompt to execute the Unix commands, enables background processes and allows usage of pipes. It also maintains a history of executed commands and background.

The command prompt takes commands from the user as a string. This string is then parsed into individual commands. If the pipe symbol("|") used in the input then after parsing the input string, pipes are set up between the two commands for command chaining. Then, each command in the commands[] array is iterated through a loop while checking whether the command runs in the background.

A child process is created for command execution using fork() method. If the command is not the first one in the chain of commands then the child process takes its input from the previous pipe and if it is not the last one, its output is sent to the next command through the next pipe. After the child process finishes, the unused pipe ends are closed in the parent process.

After this we check whether there is a "&" at the end of the command, if it exists then the command is run in the background, or else the command is run in the foreground.

Finally the execution time and command history is recorded.

Contributions:

- Varun Srivastava: parse_user_command, input_user_command, background, error checks
- Dhruv Jain: launch, background, history, error checks

Commands that will not be supported:

1. Built-in Shell Commands :

Commands like `cd`, `export`, and `alias` are shell built-in commands. These will not work unless you explicitly implement them. For example:

cd: We can't change the current working directory of the shell process.

export: Involves setting environment variables in the shell's process,.

2. Input/Output Redirection :

Commands that involve input/output redirection require handling file descriptors using `open()`, `close()`, etc. Without this, redirection won't work.

E.g - `<` , `>`

3. Job Control :

Advanced job control features like suspending and resuming jobs require managing process groups, signals (`SIGSTOP`, `SIGCONT`), and terminal control.

E.g - `fg` , `bg`, `jobs`

4. Signal Handling :

We would have to implement custom signal handling to manage interrupts (`SIGINT`, `SIGTERM`) within the shell itself.

E.g - `Ctrl+Z`

5. Shell Scripts:

Won't support running shell scripts without writing a parser to interpret shell script syntax.

6. Environment Variable Expansion:

`$` in code and as per assignment regulations would start processing as background processes.

E.g - `$HOME`, `$PATH`